Lab 08
Curves and Surfaces Modeling

Objectives

- We will learn modelling using Bezier Curves, Bezier Surfaces and Sweep (Curve) Representations.

Programming Exercises (90 minutes)

Task 1: Using Bezier Curves *(20 minutes)*

Task 1A: Setting up

Let's first declare the `MyBezierLine` class. Part of `CGLab08.hpp` should have these lines below:

```
class MyBezierLine
{
 public:
    MyBezierLine() { }
    ~MyBezierLine() { }
    void setup(const GLfloat* controlPoints,
               GLint uOrder);
    void draw(GLenum draw_mode = GL_LINE, //or GL_POINT
              GLint ures = 100);
    void drawControlPoints();

 private:
    GLint uorder;
    const GLfloat* controlpoints;
};

class MyVirtualWorld
{
 public:
     MyBezierLine mybezierline;

    void draw()
    {
     glColor3f(1.0f, 0.0f, 1.0f);
     mybezierline.draw(GL_LINE); //or GL_POINT
     glColor3f(0.0f, 1.0f, 1.0f);
     mybezierline.drawControlPoints();
    }

    void tickTime()
    {
     //do nothing, reserved for doing animation
    }

    //for any one-time only initialization of the
    //   virtual world before any rendering takes place
    //   BUT after OpenGL has been initialized
```

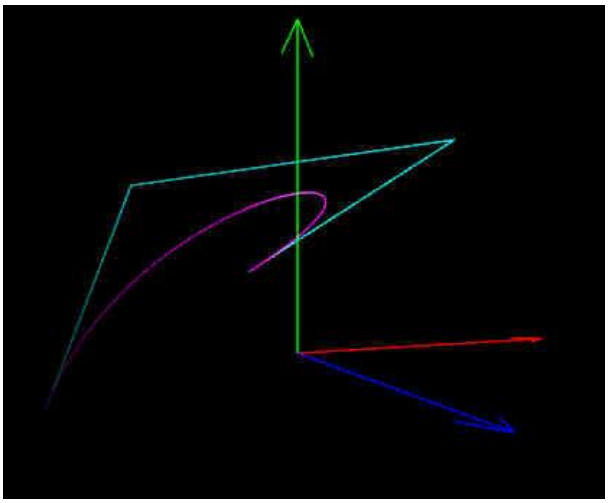
```
    void init()
    {
      static GLfloat controlpoints1[] =
               { -15.0f,  0.0f, 10.0f,
                 -10.0f, 10.0f,  0.0f,
                  15.0f, 15.0f, -5.0f,
                   0.0f,  5.0f,-10.0f
               };
      mybezierline.setup(controlpoints1, 4);
    }
};
```

The CGLab08.cpp should be as below:

```
/*
 TCG6223 Computer Graphics
 FIST, Multimedia University

 CGLab08.cpp

 Objective: Lab08 on Drawing Curves and Surfaces
*/

#include <GL/glut.h>
#include "CGLab08.hpp"

using namespace CGLab08;

void MyBezierLine::setup(const GLfloat* controlPoints,
                         GLint uOrder)
{
 controlpoints = controlPoints;
 uorder = uOrder;

 //setup the bezier
 glMap1f(GL_MAP1_VERTEX_3,   //to be generated data
         0.0f,               //lower u range
         1.0f,               //higher u range
         3,                  //u control point stride in array
         uorder,         //order of u or number of u control point
         controlpoints); //the control points array

 //enabling bezier evaluator
 glEnable(GL_MAP1_VERTEX_3);
}

void MyBezierLine::draw(GLenum draw_mode, //GL_LINE or GL_POINT
                        GLint ures)
{
 //setting the 1D grid map containing ures points
 //map to u range 0.0 - 1.0
 glMapGrid1f(ures, 0.0f, 1.0f);

 //evaluate the bezier surface
 glEvalMesh1(draw_mode, 0, ures);
}
```

```
void MyBezierLine::drawControlPoints()
{
 //draw the control points
 int index = 0;
 glBegin(GL_LINE_STRIP);
    for(int i=0; i < uorder; ++i)
    {
       glVertex3fv(controlpoints + index );
       index += 3;
    }
 glEnd();
}
```

Compile and run the program, and the output should be:



Task 1B: Modify and Observe

Modify the control points in `MyVirtualWorld::init()` function and run the program again to see the effect.

Task 2: Using Bezier Surfaces *(20 minutes)*

Task 2A: Setting up

Let's now add the `MyBezierSurface` class to the `CGLab08.hpp` as below:

```
class MyBezierSurface
{
 public:
    MyBezierSurface() { }
    ~MyBezierSurface() { }
    void setup(const GLfloat* controlPoints,
               GLint uOrder, GLint vOrder,
               const bool autonormal = false);
    void draw(GLenum draw_mode = GL_FILL, //or GL_LINE or GL_POINT
              GLint ures = 100,GLint vres = 100);
    void drawControlPoints();

 private:
    GLint uorder, vorder;
    const GLfloat* controlpoints;
};
```

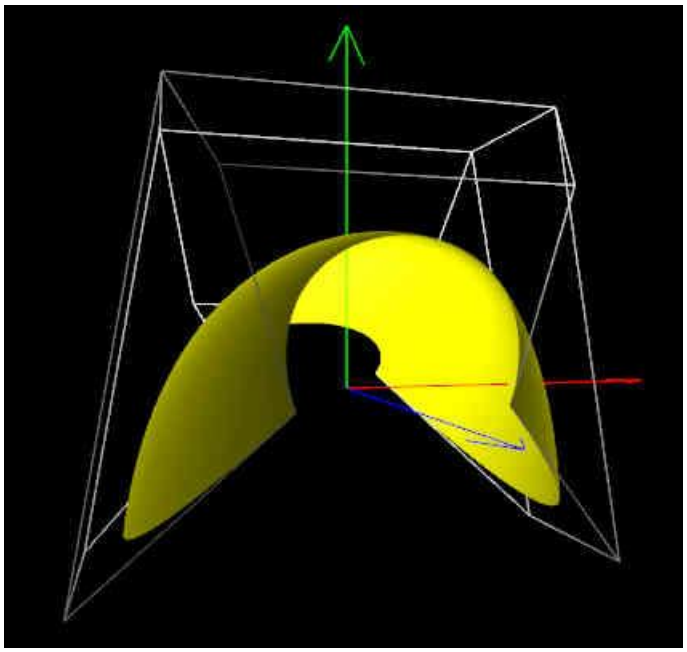
```
class MyVirtualWorld
{
 public:
    //MyBezierLine mybezierline;
     MyBezierSurface mybeziersurface;

    void draw()
    {
     //glColor3f(1.0f, 0.0f, 1.0f);
     //mybezierline.draw(GL_LINE); //or GL_POINT
     //glColor3f(0.0f, 1.0f, 1.0f);
     //mybezierline.drawControlPoints();

      glColor3f(1.0f, 1.0f, 0.0f);
      mybeziersurface.draw(GL_FILL); //or GL_LINE or GL_POINT
      glColor3f(1.0f, 1.0f, 1.0f);
      mybeziersurface.drawControlPoints();
    }

    void tickTime()
    {
     //do nothing, reserved for doing animation
    }

    //for any one-time only initialization of the
    //   virtual world before any rendering takes place
    //   BUT after OpenGL has been initialized
    void init()
    {
      static GLfloat controlpoints2[] =
                {  -5.0f,  0.0f, 10.0f,
                   -8.0f, 10.0f, 10.0f,
                    8.0f, 10.0f, 10.0f,
                    5.0f,  0.0f, 10.0f,

                  -15.0f,-10.0f,  5.0f,
                  -10.0f, 15.0f,  5.0f,
                   10.0f, 15.0f,  5.0f,
                   15.0f,-10.0f,  5.0f,

                  -15.0f,-10.0f, -5.0f,
                  -10.0f, 15.0f, -5.0f,
                   10.0f, 15.0f, -5.0f,
                   15.0f,-10.0f, -5.0f,

                   -5.0f,  0.0f,-10.0f,
                   -8.0f,  5.0f,-10.0f,
                    8.0f,  5.0f,-10.0f,
                    5.0f,  0.0f,-10.0f
                };
      mybeziersurface.setup(controlpoints2, 4, 4);    }
};
```

Add these into the `CGLab08.cpp`:

```
void MyBezierSurface::setup(const GLfloat* controlPoints,
                            GLint uOrder, GLint vOrder,
                            const bool autonormal)
{
 controlpoints = controlPoints;
 uorder = uOrder;
 vorder = vOrder;

 //setup the bezier
 glMap2f(GL_MAP2_VERTEX_3,  //to be generated data, surface vertex
         0.0f,              //lower u range
         1.0f,              //higher u range
         3,                 //u control point stride in array
         uorder,            //order of u or number of u control point
         0.0f,              //lower v range
         1.0f,              //higher v range
         3 * uorder,        //v control point stride in array
         vorder,            //order of v or number of v control point
         controlpoints); //the control points array

 //enabling bezier evaluator
 glEnable(GL_MAP2_VERTEX_3);

 //enabling auto generation of normals for use with lighting
 if (autonormal) glEnable(GL_AUTO_NORMAL);
}

void MyBezierSurface::draw(GLenum draw_mode, //GL_FILL or GL_LINE or GL_POINT
                           GLint ures,GLint vres)
{
 glDisable(GL_CULL_FACE);
 //setting the 2D grid map containing ures * vres points
 //map to u range 0.0 - 1.0, v range 0.0 - 1.0
 glMapGrid2f(ures, 0.0f, 1.0f, vres, 0.0f, 1.0f);

 //evaluate the bezier surface
 glEvalMesh2(draw_mode, 0, ures, 0, vres);
 glEnable(GL_CULL_FACE);
}

void MyBezierSurface::drawControlPoints()
{
 //draw the control points
 int index = 0;
 for(int j=0; j < vorder; ++j)
 {
    glBegin(GL_LINE_STRIP);
       for(int i=0; i < uorder; ++i)
       {
          glVertex3fv(controlpoints + index );
          index += 3;
       }
    glEnd();
 }
```

```
 for(int i=0; i < uorder; ++i)
 {
    index = 3*i;
    glBegin(GL_LINE_STRIP);
       for(int j=0; j < vorder; ++j)
       {
          glVertex3fv(controlpoints + index );
          index += uorder*3;
       }
    glEnd();
 }
}
```

Compile and run the program, and the output should be:



Task 2B: Modify and Observe

Modify the control points in `MyVirtualWorld::init()` function and run the program again to see the effect.

Task 3: Using Sweep Surfaces *(20 minutes)*

Task 3A: Setting up

Let's first declare the `MySweepSurface` class. Part of `CGLab08.hpp` should have these lines below:

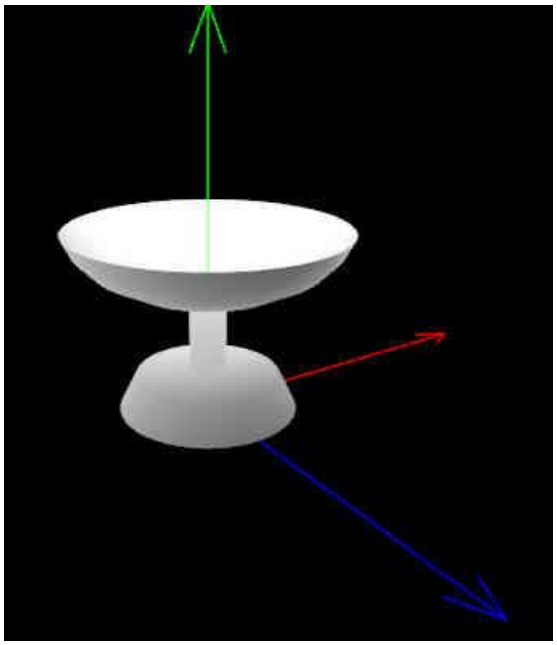
```
//sweep about axis-y
class MySweepSurface
{
 public:
    MySweepSurface() { }
    ~MySweepSurface()
    {
     if (surfacepoints) delete[] surfacepoints;
    }
```

```
    //note: size of profilePoints = 3*numOfProfilePoints
    void setup(const GLfloat* profilePoints,
               GLint numOfProfilePoints,
               GLfloat degreeStart,
               GLfloat degreeEnd,
               GLfloat degreeStep );
    void draw();

 private:
    const GLfloat* profilepoints;
    GLint numofprofilepoints;
    GLfloat *surfacepoints;
    GLint numofsurfacepoints;
    GLint numofverticallines;
};

class MyVirtualWorld
{
 public:
    MySweepSurface mysweepsurface;

    void draw()
    {
      glColor3f(1.0f, 1.0f, 1.0f);
      mysweepsurface.draw();
    }

    void tickTime()
    {
     //do nothing, reserved for doing animation
    }

    //for any one-time only initialization of the
    //   virtual world before any rendering takes place
    //   BUT after OpenGL has been initialized
    void init()
    {
      static GLfloat profilepoints[] =
                {  5.0f,  0.0f,  0.0f,
                   4.0f,  2.0f,  0.0f,
                   1.0f,  2.0f,  0.0f,
                   1.0f,  6.0f,  0.0f,
                   4.0f,  7.0f,  0.0f,
                   6.0f,  8.0f,  0.0f,
                   7.0f,  9.0f,  0.0f,
                   7.5f, 10.0f,  0.0f
                };
      mysweepsurface.setup(profilepoints, 8, 0, 360, 5);
    }
};
```

Add these into the `CGLab08.cpp`:

```
//note: size of profilePoints = 3*numOfProfilePoints
void MySweepSurface::setup(const GLfloat* profilePoints,
                           GLint numOfProfilePoints,
                           GLfloat degreeStart,
                           GLfloat degreeEnd,
                           GLfloat degreeStep )
{
 profilepoints = profilePoints;
 numofprofilepoints = numOfProfilePoints;

 if (surfacepoints) delete[] surfacepoints;

 numofverticallines = 1 + static_cast<int>(floor((degreeEnd - degreeStart)
                                                 /degreeStep));
 numofsurfacepoints = numofprofilepoints
                    * numofverticallines;
 surfacepoints = new GLfloat[3 * numofsurfacepoints];

 int surfptsindex = 0;
 GLfloat degree = degreeStart;
 for (int i=0; i<numofverticallines; ++i, degree+=degreeStep)
 {
    GLfloat radian = degree*M_PI/180.0;
    GLfloat c = cos(radian);
    GLfloat s = sin(radian);
    GLfloat x,y,z;
    for (int j=0, profileindex=0; j<numofprofilepoints; ++j)
    {
       surfacepoints[surfptsindex]   =  c*profilepoints[profileindex]
                                      + s*profilepoints[profileindex+2];
       surfacepoints[surfptsindex+1] =    profilepoints[profileindex+1];
       surfacepoints[surfptsindex+2] = -s*profilepoints[profileindex]
                                      + c*profilepoints[profileindex+2];
       profileindex += 3;
       surfptsindex += 3;
    }
 }
}

void MySweepSurface::draw()
{
 glDisable(GL_CULL_FACE);
 int index = 0;
 for (int i=0; i<numofverticallines-1; ++i)
 {
    glBegin(GL_QUAD_STRIP);
       for (int j=0; j<numofprofilepoints; ++j)
       {
          glVertex3fv( surfacepoints+index );
          glVertex3fv( surfacepoints+index+3*numofprofilepoints );
          index+=3;
       }
    glEnd();
 }
 glEnable(GL_CULL_FACE);
}
```

Compile and run the program, the output should be:



Task 3B: Modify and Observe

Modify the profile points in `MyVirtualWorld::init()` function and run the program again to see the effect.

Task 3C: Change Sweep profile points

Take note that in the example given above, the sweep angle is from 0 degree to 360 degrees, with an interval of 5 degree. There are 8 points altogether in the profile curve. Try to change these values and see the effect.

You can also try changing the profile points to create different kinds of "sweep" objects!

Task 4: Combining Curves *(Homework)*

Create two Bezier Curves that are joined smoothly at one point, see the lecture notes on the condition to join two curves smoothly.

HAVE FUN !!

Extra Exercises

VERY IMPORTANT

BEFORE coming to the next lab, you MUST:

1. COMPLETE all the exercises in this lab!

2. DO the extra exercises below (if there are any)!

3. READ the lab instructions!

4. PREPARE files for next lab!

1. Play around with the `MyVirtualWorld::init()` function in all the examples above to build round and curve shape models.

2. Play around with Bezier Curve and Bezier Surfaces with different *degree* of curves.

3. Create two Bezier Surfaces that are joined smoothly at one edge, see the lecture notes on the condition to join surfaces smoothly.

4. Try to create other objects using profile line.

5. There is another modelling technique called **Extrusion** in which you create a cross-section of an object (something like a 2D profile cross-section plane rather than a 1D profile line) and drag it along a 3D path to create a complex object. Research on this and try to create some of such objects.

* END OF LAB *